

03 容器与 STL 对照

目标:把 C++ STL 12 个常用容器映射到 Java 对应物,知道常用 API 怎么翻译。
重点深讲:ArrayList / ArrayDeque / PriorityQueue / HashMap / HashSet / LinkedList。
装箱小框(从 02 迁入):ArrayList<Integer> 必须用包装类,Integer 缓存池有陷阱。

一、开篇定调:为什么这一章是核心

Java 刷题,90% 的时间都在和容器打交道。STL 熟不熟直接决定 Java 刷题顺不顺。这一章的设计思路是:

- 12 个常用 STL 容器 → Java 对应物:一张表速查
- 6 个深讲:ArrayList / ArrayDeque / PriorityQueue / HashMap / HashSet / LinkedList,每个都给 API 列表 + 最小代码 + 1-2 个坑点
- 6 个简讲:Stack(别用)/ TreeMap / TreeSet / Queue(接口)/ Deque(接口)/ Vector(别用),一行对照 + 一行代码
- pair 单独展开:Java 没有内建 pair,这里讲清楚 3 种替代写法
- 选型决策树:忘记选哪个?翻决策树

读完这一章,看到任何 C++ STL 容器写法,你都能立刻翻译成 Java 并知道常用 API。

二、12 容器总览对照表(5 列速查)

使用方式:看到左边 C++ 容器 → 找到 Java 对应 → 看"常用 API 映射"列 → 看"深度"列决定要不要细读这一节。

C++ STL	Java	关键差异	常用 API 映射	深度
vector<T>	ArrayList<T>	只能装对象,基本类型用 Integer	push_back→add / size→size / empty→isEmpty / []→get/set	深 ★
list<T>	LinkedList<T>	也可当队列/栈,但慢,推荐 ArrayDeque	push_back→add / push_front→addFirst / front→peek	中
stack<T>	ArrayDeque<T> 当栈用	java.util.Stack 性能差且过时,别用	push→push / top→peek / pop→pop	深 ★
queue<T>	ArrayDeque<T> / LinkedList	默认 ArrayDeque	push→offer / front→peek / pop→poll	深 ★
priority_queue<T>	PriorityQueue<T>	默认小顶堆(C++ 默认大顶堆),反的!	push→offer / top→peek / pop→poll	深 ★
deque<T>	ArrayDeque<T>	滑动窗口标配	push_back→addLast / push_front→addFirst / []→peekFirst/peekLast	深 ★

unordered_map<K,V>	HashMap<K,V>	允许 null key/value	[]→get/put / find→containsKey / count→size	深 ★
unordered_set<T>	HashSet<T>	基于 HashMap	insert→add / find→contains / count→size	中
无直接对应	LinkedHashMap<K,V>	保持插入顺序或访问顺序,LRU 缓存标准答案	get→get / put→put / 构造 new LinkedHashMap<>(cap , 0.75f, true)	中
map<K,V>	TreeMap<K,V>	红黑树,有序	[]→get/put / lower_bound→lowerKey / upper_bound→higherKey	简
set<T>	TreeSet<T>	红黑树,有序	insert→add / find→contains / lower_bound→lower	简
pair<T1,T2>	int[] / Map.Entry / 自定义类	Java 无内建 pair,见下方	见下方	简
sort	Arrays.sort / Collections.sort		sort→sort / stable_sort→Arrays.sort (对象)	中

13 个容器分级(12 + LinkedHashMap):

- 深讲 6 个(本篇核心):ArrayList / ArrayDeque / PriorityQueue / HashMap / HashSet / LinkedList
- 简讲 7 个(在第七节一行对照):Stack / TreeMap / TreeSet / Queue / Deque / Vector / LinkedHashMap(LRU 缓存特例)

三、装箱小框(必看,看完对照表立刻看)

装箱小框

>

Integer a = 5; int b = a; 自动装箱/拆箱,ArrayList<Integer> 必须用包装类(因为 Java 泛型不支持基本类型)。

>

Integer 缓存池陷阱:Integer a = 100; Integer b = 100; a == b 是 true(在 -128~127 范围内,自动装箱用缓存池);但 a = 200; b = 200; a == b 是 false(超出缓存池范围,new Integer(200))。

>

永远用 equals。永远用 equals。永远用 equals。说三遍。

```
Integer a = 100, b = 100;
System.out.println(a == b); // true(缓存池范围内)
Integer c = 200, d = 200;
System.out.println(c == d); // false(超出缓存池,new 出来的)
System.out.println(c.equals(d)); // true(永远用 equals)
```



```
// C++ 没有这个问题,int 是基本类型,== 就是值比较
int a = 100, b = 100;
cout << (a == b); // true,无误
```

关键提醒:这个坑在 06 第 3 坑还会再出现一次。刷题时 Integer / Long / String 比较,一律 .equals()。这个坑是 C++ 选手转 Java 的第 1 大坑。

四、深讲 1:ArrayList<E>(对应 vector)

概述

ArrayList<E> 是 Java 最常用的容器,对应 C++ vector<T>。底层是动态数组,默认容量 10,扩容为原来的 1.5 倍。

关键特性:

- 只能装对象,基本类型必须用包装类(ArrayList<Integer>,不能 ArrayList<int>)
- 随机访问 O(1),末尾添加均摊 O(1),中间插入/删除 O(n)
- 不线程安全(要线程安全用 Collections.synchronizedList 或 CopyOnWriteArrayList,刷题用不到)

API 速查

方法	说明
add(E e)	末尾添加
add(int index, E e)	在 index 位置插入
get(int index)	读 index 位置
set(int index, E e)	改 index 位置
remove(int index)	按下标删
remove(Object o)	按值删(删第一个匹配)
size() / isEmpty() / clear()	长度 / 判空 / 清空
contains(Object o)	是否包含
toArray()	转 Object[]
Arrays.asList(T... a)	返回定长 List<T>,不能 add/remove
new ArrayList<>(Arrays.asList(...))	转成可变 List(常用)

最小代码:读 N 个数 → 排序 → 输出



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 构造:三种常用方式
        ArrayList<Integer> list1 = new ArrayList<>();    // 空列表
        ArrayList<Integer> list2 = new ArrayList<>(Arrays.asList(1, 2, 3)); // 一次性初始化
        ArrayList<Integer> list3 = new ArrayList<>(list2);    // 拷贝构造

        // 2. 增删改查
        list1.add(10);    // 末尾添加
        list1.add(0, 5);    // index 0 插入 5
        list1.set(0, 7);    // 改 index 0 为 7
        int x = list1.get(0);    // 读 index 0
        list1.remove(0);    // 按下标删
        list1.remove(Integer.valueOf(10)); // 按值删(注意要传 Integer,不是 int)

        // 3. 遍历(三种方式)
        for (int i = 0; i < list1.size(); i++) {    // 经典 for
            System.out.println(list1.get(i));
        }
        for (int v : list1) {    // for-each(常用)
            System.out.println(v);
        }
        list1.forEach(System.out::println);    // lambda(方法引用)

        // 4. 排序
        Collections.sort(list1);    // 升序
        list1.sort(Comparator.reverseOrder()); // 降序(Java 8 新增)

        // 5. 转数组
        Integer[] arr = list1.toArray(new Integer[0]);
    }
}

```



```
// C++ 对照
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v1;           // 空
    vector<int> v2 = {1, 2, 3}; // 初始化

    v1.push_back(10);         // 末尾添加
    v1.insert(v1.begin(), 5);  // 头部插入
    v1[0] = 7;                 // 改
    int x = v1[0];             // 读
    v1.erase(v1.begin());      // 按下标删
    auto it = find(v1.begin(), v1.end(), 10);
    if (it != v1.end()) v1.erase(it); // 按值删

    for (int i = 0; i < v1.size(); i++) cout << v1[i];
    for (int v : v1) cout << v;      // 范围 for

    sort(v1.begin(), v1.end());      // 升序
    sort(v1.rbegin(), v1.rend());    // 降序
}
```

坑点 1:基本类型必须用包装类

```
// 错误:Java 泛型不支持基本类型
ArrayList<int> list = new ArrayList<>(); // 编译报错

// 正确:用 Integer 包装类
ArrayList<Integer> list = new ArrayList<>();
list.add(5); // 5 自动装箱成 Integer.valueOf(5)
int x = list.get(0); // Integer 自动拆箱成 int
```

```
// C++ 没这个问题,vector<int> 是合法的
vector<int> v;
v.push_back(5);
int x = v[0];
```

坑点 2:Arrays.asList 返回定长 list

```
List<Integer> list = Arrays.asList(1, 2, 3);
list.add(4); // 错误:UnsupportedOperationException,Arrays.asList 返回定长

// 正确:外面包一层 ArrayList
List<Integer> mutable = new ArrayList<>(Arrays.asList(1, 2, 3));
mutable.add(4); // 正常
```

这个坑在 06 第 5 坑还会再出现。刷题时 Arrays.asList(...) 当参数传可以,不要拿去 add/remove。

何时用 ArrayList



- 90% 场景的默认选择
- 随机访问多、末尾增删多 → 用 ArrayList
- 频繁中间插入/删除 → 考虑 LinkedList(但其实也慢,见 LinkedList 节)

五、深讲 2:ArrayDeque<E>(对应 stack / queue / deque)

概述

ArrayDeque<E> 是 Java 中同时实现栈、队列、双端队列的容器,基于循环数组,性能在三种场景下都是最优。刷题时遇到栈/队列/双端队列,99% 情况用 ArrayDeque。

关键特性:

- 当栈用:push/pop/peek
- 当队列用:offer/poll/peek
- 当双端队列用:addFirst/addLast/peekFirst/peekLast/pollFirst/pollLast
- 性能优于 Stack(继承 Vector,有同步开销)和 LinkedList(有节点对象开销)
- 不能存 null(会抛 NullPointerException)

API 速查

当栈用:

方法	说明
push(E e)	压栈
pop()	出栈
peek()	看栈顶(不出栈)

当队列用:

方法	说明
offer(E e)	入队(返回 boolean,容量满则 false)
poll()	出队(返回 null 表示空)
peek()	看队首(返回 null 表示空)

当双端队列用:

方法	说明
addFirst(E e) / addLast(E e)	头/尾添加
peekFirst() / peekLast()	头/尾查看
pollFirst() / pollLast()	头/尾出队
removeFirst() / removeLast()	头/尾删除(空时抛异常)

最小代码:滑动窗口(当单调队列用)



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 当栈用:有效括号(LeetCode 20)
        ArrayDeque<Character> stack = new ArrayDeque<>();
        String s = "()[]{}";
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') {
                stack.push(c);    // 压栈
            } else {
                if (stack.isEmpty() || !match(stack.pop(), c)) {
                    System.out.println("invalid");
                    return;
                }
            }
        }
        System.out.println(stack.isEmpty() ? "valid" : "invalid");

        // 当队列用:广度优先遍历
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        queue.offer(1);
        queue.offer(2);
        while (!queue.isEmpty()) {
            int x = queue.poll(); // 出队
            System.out.println(x);
        }

        // 当双端队列用:滑动窗口最大值(LeetCode 239 关键)
        ArrayDeque<Integer> deque = new ArrayDeque<>();
        int[] nums = {1, 3, -1, -3, 5, 3, 6, 7};
        int k = 3;
        for (int i = 0; i < nums.length; i++) {
            // 1. 移除窗口外元素
            while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
                deque.pollFirst();
            }
            // 2. 维护单调性(单调递减,队首最大)
            while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
                deque.pollLast();
            }
            deque.offerLast(i); // 入队
            // 3. 窗口形成后记录答案
            if (i >= k - 1) {
                System.out.println(nums[deque.peekFirst()]); // 当前窗口最大值
            }
        }

        static boolean match(char l, char r) {
            return (l == '(' && r == ')') || (l == '[' && r == ']') || (l == '{' && r == '}');
        }
    }
}

```



```
// C++ 对照
#include <stack>
#include <queue>
#include <deque>
using namespace std;

int main() {
    // 栈
    stack<char> s;
    s.push('a'); s.pop(); char t = s.top();

    // 队列
    queue<int> q;
    q.push(1); q.push(2);
    int x = q.front(); q.pop();

    // 双端队列
    deque<int> dq;
    dq.push_back(1); dq.push_front(2);
    int a = dq.front(); int b = dq.back();
    dq.pop_back(); dq.pop_front();
}
```

坑点 1:不能存 null

```
ArrayDeque<Integer> deque = new ArrayDeque<>();
deque.push(null); // 错误:NullPointerException
deque.offer(null); // 错误:NullPointerException
```

```
// C++ 的 deque 可以存 0 指针 Java 不行
deque<int*> dq;
dq.push_back(nullptr); // C++ OK
```

坑点 2:别再用 java.util.Stack

```
// 错误:java.util.Stack 继承 Vector,有 synchronized 同步开销
Stack<Integer> stack = new Stack<>();
stack.push(1);
stack.pop();

// 正确:用 ArrayDeque 替代
ArrayDeque<Integer> deque = new ArrayDeque<>();
deque.push(1);
deque.pop();
```

原因:Stack 是 JDK 1.0 时代的设计,继承 Vector 导致所有方法都有 synchronized 锁,性能差且设计过时。刷题时永远用 ArrayDeque 当栈。这个坑在 06 第 1 坑会再出现。

何时用 ArrayDeque

- 栈/队列/双端队列 → 默认用 ArrayDeque



- 滑动窗口最大值(BFS 时的双端操作)→ 必用
- 不用 LinkedList(慢),不用 Stack(过时)

六、深讲 3:PriorityQueue<E>(对应 priority_queue)

概述

PriorityQueue<E> 是 Java 的堆实现,基于小顶堆(注意,反 C++!)

关键特性:

- 默认小顶堆(堆顶是最小元素),C++ priority_queue 默认大顶堆(堆顶是最大元素),反的!
- 自定义比较器可改成大顶堆
- 底层是最小二叉堆(非线程安全的 PriorityQueue),不是 PriorityBlockingQueue
- 不能存 null
- 时间复杂度:插入/删除 $O(\log n)$,查堆顶 $O(1)$

API 速查

方法	说明
offer(E e)	插入(同 add)
peek()	看堆顶(不删)
poll()	出堆顶
size() / isEmpty()	长度 / 判空
new PriorityQueue<>((a, b) -> b - a)	大顶堆,反转比较器
PriorityQueue<>(Collections.reverseOrder())	大顶堆(标准写法)

最小代码:LeetCode 347 前 K 个高频元素(大顶堆)



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 默认小顶堆(堆顶最小)
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        minHeap.offer(5);
        minHeap.offer(1);
        minHeap.offer(3);
        System.out.println(minHeap.peek()); // 1(堆顶最小)
        System.out.println(minHeap.poll()); // 1
        System.out.println(minHeap.poll()); // 3

        // 2. 大顶堆(堆顶最大)—— 两种写法
        PriorityQueue<Integer> maxHeap1 = new PriorityQueue<>((a, b) -> b - a);
        PriorityQueue<Integer> maxHeap2 = new PriorityQueue<>(Comparator.reverseOrder());
        maxHeap1.offer(5);
        maxHeap1.offer(1);
        maxHeap1.offer(3);
        System.out.println(maxHeap1.poll()); // 5(堆顶最大)

        // 3. 实际应用:前 K 个高频元素(LeetCode 347 简化版)
        int[] nums = {1, 1, 1, 2, 2, 3};
        int k = 2;
        // 统计频次
        Map<Integer, Integer> freq = new HashMap<>();
        for (int n : nums) {
            freq.put(n, freq.getOrDefault(n, 0) + 1);
        }
        // 小顶堆维护前 K 高频
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            pq.offer(new int[]{e.getKey(), e.getValue()});
            if (pq.size() > k) {
                pq.poll(); // 弹出频次最低的
            }
        }
        // 输出前 K 高频元素
        List<Integer> result = new ArrayList<>();
        while (!pq.isEmpty()) {
            result.add(pq.poll()[0]);
        }
        System.out.println(result); // [1, 2] 或 [2, 1](频次同为 2 时顺序不定)
    }
}

```



```
// C++ 对照
#include <queue>
#include <unordered_map>
#include <vector>
using namespace std;

int main() {
    // C++ 默认大顶堆(堆顶最大)
    priority_queue<int> maxHeap;
    maxHeap.push(5); maxHeap.push(1); maxHeap.push(3);
    cout << maxHeap.top(); // 5
    maxHeap.pop();

    // C++ 小顶堆:用 greater
    priority_queue<int, vector<int>, greater<int>> minHeap;
    minHeap.push(5); minHeap.push(1);
    cout << minHeap.top(); // 1
}
```

坑点 1:默认小顶堆,记得反转

```
// 错:以为默认大顶堆,直接 poll() 得到最小值(不是最大)
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(5); pq.offer(1); pq.offer(3);
pq.poll(); // 1,不是你以为的 5

// 正确:要大顶堆必须反转
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Comparator.reverseOrder());
maxPQ.poll(); // 5
```

```
// C++ 选手最常踩这个坑:Java 反着
priority_queue<int> pq; // 大顶堆
pq.top(); // 5
```

坑点 2:不能存 null

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(null); // 错误:NullPointerException
```

自定义类的优先级队列



```
// 自定义类必须实现 Comparable 接口,或者传 Comparator
class Student {
    int score;
    String name;
    Student(int s, String n) { score = s; name = n; }
}

// 写法 1:类实现 Comparable
class Student implements Comparable<Student> {
    int score;
    public int compareTo(Student o) { return this.score - o.score; }
}

PriorityQueue<Student> pq1 = new PriorityQueue<>();

// 写法 2:传 Comparator(lambda 写法,Java 8)
PriorityQueue<Student> pq2 = new PriorityQueue<>((a, b) -> a.score - b.score);
```

何时用 PriorityQueue

- Top K 问题(前 K 大/小)→ 必用
- 堆排序 → 用 PriorityQueue 简化
- 合并 K 个有序流 → 必用
- Dijkstra 最短路 → 必用

七、深讲 4:HashMap<K, V>(对应 unordered_map)

概述

HashMap<K, V> 是 Java 的哈希表实现,对应 C++ unordered_map<K, V>。刷题出现频率前 3,LeetCode 几乎一半的题要用到。

关键特性:

- 允许 1 个 null key 和多个 null value
- 不保证元素顺序
- 线程不安全(要线程安全用 ConcurrentHashMap,刷题用不到)
- 底层:数组 + 链表 + 红黑树(Java 8+,链表长度 > 8 时转红黑树)
- 时间复杂度:get/put 平均 O(1),最坏 O(log n)(红黑树退化时)

API 速查

方法	说明
put(K, V)	插入/更新
get(K)	取值(无 key 返回 null)
containsKey(K)	是否存在 key
containsValue(V)	是否存在 value(O(n))
remove(K)	删除 key
size() / isEmpty() / clear()	长度 / 判空 / 清空
keySet()	所有 key 的 Set
values()	所有 value 的 Collection



entrySet()	所有 (K, V) 的 Set(遍历常用)
getOrDefault(K, defaultValue)	无 key 时返回默认值(Java 8 必备,见 04)
computeIfAbsent(K, k -> ...)	无 key 时计算并 put(Java 8 必备,见 04)
merge(K, V, BiFunction)	合并(Java 8 必备,见 04)

最小代码:LeetCode 1 两数之和(单遍 HashMap)

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 基本操作
        Map<String, Integer> map = new HashMap<>();
        map.put("a", 1);
        map.put("b", 2);
        map.get("a"); // 1
        map.getOrDefault("c", 0); // 0(无 key 返回默认)
        map.containsKey("a"); // true
        map.put("a", 10); // 更新,key 已存在则覆盖

        // 2. 遍历(三种方式)
        for (Map.Entry<String, Integer> e : map.entrySet()) { // entrySet(推荐)
            System.out.println(e.getKey() + " = " + e.getValue());
        }
        for (String k : map.keySet()) { // keySet
            System.out.println(k + " = " + map.get(k));
        }
        map.forEach((k, v) -> System.out.println(k + " = " + v)); // lambda

        // 3. LeetCode 1 两数之和(经典模板)
        int[] nums = {2, 7, 11, 15};
        int target = 9;
        Map<Integer, Integer> idx = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (idx.containsKey(need)) {
                System.out.println "[" + idx.get(need) + ", " + i + "]" ;
                return;
            }
            idx.put(nums[i], i);
        }
    }
}
```



```
// C++ 对照
#include <unordered_map>
using namespace std;

int main() {
    unordered_map<string, int> m;
    m["a"] = 1;          // 插入/更新
    m["b"] = 2;
    int x = m["a"];       // 取(无 key 会插入默认值.坑)
    x = m.at("a");        // 取(无 key 抛异常)
    m.count("a");         // 是否存在
    m.erase("a");        // 删除

    // 遍历
    for (auto& [k, v] : m) cout << k << " = " << v; // C++17
    for (auto it = m.begin(); it != m.end(); ++it) {
        cout << it->first << " = " << it->second;
    }
}
```

LeetCode 模式完整版:两数之和(可抄直接提交)

上面是 Main 类的本地测试版,LeetCode 提交要改成 Solution 类:

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> idx = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (idx.containsKey(need)) {
                return new int[]{idx.get(need), i};
            }
            idx.put(nums[i], i);
        }
        return new int[0]; // 题目保证有解,实际不会走到
    }
}
```

Java 8 必备:computeIfAbsent 建邻接表(LeetCode 49 字母异位词分组等题必用)

场景:把字符串按某种 key 分组(比如按字符出现次数排序后的字符串),传统写法要先 getOrDefault 再 put,又丑又慢。computeIfAbsent 一行搞定:

```
// LeetCode 49 字母异位词分组(简化版)
Map<String, List<String>> groups = new HashMap<>();
for (String s : strs) {
    char[] ca = s.toCharArray();
    Arrays.sort(ca);          // 把字符排序后当 key
    String key = new String(ca);
    groups.computeIfAbsent(key, k -> new ArrayList<>()).add(s); // ★ 一行建邻接表
}
System.out.println(new ArrayList<>(groups.values()));
// 输出: [[ "eat", "tea", "ate"], [ "tan", "nat"], [ "bat"]]
```



C++ 对照:

```
// C++ 没有 computeIfAbsent,需要先 find 再 insert,5 行代码
auto it = groups.find(key);
if (it == groups.end()) groups[key] = vector<string>{};
groups[key].push_back(s);
```

记忆口诀:computeIfAbsent(key, k -> 新值) = 没有就建,有就返回。比 getOrDefault + put 简洁一倍,是 Java 8 写邻接表的标配。

坑点 1:遍历时不能 put / remove

```
// 错误:ConcurrentModificationException
Map<Integer, Integer> map = new HashMap<>();
map.put(1, 1); map.put(2, 2);
for (Map.Entry<Integer, Integer> e : map.entrySet()) {
    if (e.getValue() == 1) {
        map.put(3, 3); // 抛 ConcurrentModificationException
    }
}

// 正确 1:用迭代器删除
Iterator<Map.Entry<Integer, Integer>> it = map.entrySet().iterator();
while (it.hasNext()) {
    Map.Entry<Integer, Integer> e = it.next();
    if (e.getValue() == 1) it.remove();
}

// 正确 2:先记下要改的,遍历完再批处理
List<Integer> toAdd = new ArrayList<>();
for (Map.Entry<Integer, Integer> e : map.entrySet()) {
    if (e.getValue() == 1) toAdd.add(99);
}
for (int k : toAdd) map.put(k, 99);
```

这个坑在 06 第 4 坑会再出现。

坑点 2:自定义 key 必须重写 hashCode / equals



```
// 错误:不重写 hashCode>equals,HashMap 找不到
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

Map<Point, Integer> map = new HashMap<>();
map.put(new Point(1, 2), 100);
map.get(new Point(1, 2)); // null!因为默认用 Object.hashCode(地址)

// 正确:重写 hashCode 和 equals
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    @Override
    public int hashCode() { return Objects.hash(x, y); }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return x == p.x && y == p.y;
    }
}
```

关键:HashMap / HashSet 的 key 必须正确实现 hashCode 和 equals,否则 contains / get 永远返回 false。

何时用 HashMap

- 出现频率映射(字符、数组元素)→ 必用
- 两数之和、子数组和、异位词分组 → 必用
- 图的邻接表(配合 computeIfAbsent)→ 必用
- 90% 哈希场景 → 默认用 HashMap

八、深讲 5:HashSet<E>(对应 unordered_set)

概述

HashSet<E> 是 Java 的哈希集合,对应 C++ unordered_set<T>。底层就是 HashMap,只用了 key 部分。

关键特性:

- 元素不重复
- 不保证顺序
- 允许 1 个 null 元素
- 线程不安全

API 速查

方法	说明
add(E e)	添加(已存在则返回 false)
contains(E e)	是否存在
remove(E e)	删除



size() / isEmpty() / clear()	长度 / 判空 / 清空
addAll(Collection)	并集
retainAll(Collection)	交集
removeAll(Collection)	差集

最小代码:去重 + 找交集

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 基本操作
        Set<Integer> set = new HashSet<>();
        set.add(1); set.add(2); set.add(2); // 重复加,size 仍为 2
        set.contains(1); // true
        set.remove(1);

        // 2. 去重
        List<Integer> list = Arrays.asList(1, 2, 2, 3, 3, 3);
        Set<Integer> unique = new HashSet<>(list); // [1, 2, 3]

        // 3. 找交集(LeetCode 349 两个数组的交集)
        int[] nums1 = {1, 2, 2, 1};
        int[] nums2 = {2, 2};
        Set<Integer> s1 = new HashSet<>();
        for (int n : nums1) s1.add(n);
        Set<Integer> s2 = new HashSet<>();
        for (int n : nums2) s2.add(n);
        s1.retainAll(s2); // 交集,s1 变成 {2}
        System.out.println(s1); // [2]

        // 4. 遍历
        for (int x : set) System.out.println(x);
        set.forEach(System.out::println);
    }
}
```



```
// C++ 对照
#include <unordered_set>
using namespace std;

int main() {
    unordered_set<int> s;
    s.insert(1); s.insert(2); s.insert(2); // 重复
    s.count(1); // 1(存在)
    s.erase(1);

    // 遍历
    for (int x : s) cout << x;

    // 交集(无内建,需手写)
}
```

坑点 1:自定义元素必须重写 hashCode / equals

```
// 错误:同 HashMap 的 key
Set<Point> set = new HashSet<>();
set.add(new Point(1, 2));
set.contains(new Point(1, 2)); // false!因为默认用地址比较

// 正确:重写 hashCode>equals(同 Point 类)
```

坑点 2:不能存可变对象

```
// 危险:List 的 hashCode 基于内容,改了内容 hashCode 就变,HashSet 找不回来
Set<List<Integer>> set = new HashSet<>();
List<Integer> key = new ArrayList<>(Arrays.asList(1, 2));
set.add(key);
key.add(3); // 改了内容
set.contains(key); // false!因为 hashCode 已变,桶位置不同
```

关键:HashMap / HashSet 的 key 必须是不可变对象(或至少在作为 key 期间不变)。String / Integer 是不可变的,自定义类把字段也设为 final。

何时用 HashSet

- 去重 → 必用
- 集合运算(交并差)→ 用 HashSet
- 判断存在性(常数时间)→ 用 HashSet
- 99% 哈希集合场景 → 默认用 HashSet

九、深讲 6:LinkedList<E>(对应 list,也可当 queue)

概述

LinkedList<E> 是 Java 的双向链表实现,对应 C++ list<T>。既可当 list,也可当 queue / deque。



关键特性:

- 双向链表
- 随机访问 $O(n)$ (比 ArrayList 慢)
- 头尾增删 $O(1)$
- 既实现了 List 接口,也实现了 Deque 接口
- 不线程安全

API 速查

当 list 用:

方法	说明
<code>add(E e)</code>	末尾添加
<code>add(int index, E e)</code>	指定位置插入
<code>get(int index)</code>	按下标读($O(n)$)
<code>set(int index, E e)</code>	按下标改
<code>remove(int index)</code>	按下标删
<code>remove(Object o)</code>	按值删

当 queue / deque 用:

方法	说明
<code>addFirst(E e) / addLast(E e)</code>	头/尾添加
<code>getFirst() / getLast()</code>	头/尾读
<code>removeFirst() / removeLast()</code>	头/尾删
<code>offer(E e) / poll() / peek()</code>	当队列用

最小代码:LRU 缓存(LeetCode 146 简化版)



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 当 list 用
        LinkedList<Integer> list = new LinkedList<>();
        list.add(1); list.add(2);
        list.addFirst(0); // 头插
        list.addLast(3); // 尾插
        list.get(0); // 按下标(O(n).慢)
        list.removeFirst();
        list.removeLast();

        // 当队列用
        LinkedList<Integer> queue = new LinkedList<>();
        queue.offer(1);
        queue.poll();

        // 当双端队列用
        LinkedList<Integer> deque = new LinkedList<>();
        deque.addFirst(1);
        deque.addLast(2);

        // ★ LRU 缓存(简化):用 LinkedList 维护访问顺序 —— O(n) **非最优**
        // 工业标准:用 LinkedHashMap(accessOrder=true),O(1) 实现,见对照表第 37 行 + 速查表 13 容器
        LinkedList<Integer> lru = new LinkedList<>();
        int capacity = 3;
        int[] access = {1, 2, 3, 2, 4, 1};
        for (int x : access) {
            lru.remove(Integer.valueOf(x)); // 先移除旧的
            lru.addLast(x); // 加到尾部
            if (lru.size() > capacity) {
                lru.removeFirst(); // 淘汰最久未用
            }
            System.out.println(lru);
        }
    }
}

```

```

// C++ 对照
#include <list>
using namespace std;

int main() {
    list<int> lst;
    lst.push_back(1);
    lst.push_front(0);
    lst.insert(lst.begin(), 99);
    lst.pop_back();
    lst.pop_front();

    // C++ list 只能当 list,不能当 queue(要 queue 用 std::queue 包一层)
}

```



坑点 1:比 ArrayList 慢(随机访问)

```
// 慢:LinkedList 随机访问 O(n)
LinkedList<Integer> list = new LinkedList<>();
for (int i = 0; i < 100000; i++) list.add(i);
int x = list.get(50000); // 从头遍历 50000 次

// 快:ArrayList 随机访问 O(1)
ArrayList<Integer> arr = new ArrayList<>();
for (int i = 0; i < 100000; i++) arr.add(i);
int y = arr.get(50000); // 直接定位
```

常见误解:"链表增删快,所以增删多用 LinkedList"。错! ArrayList 末尾增删 O(1),只有中间插入/删除才有差别,而且实际场景 ArrayList 的 CPU 缓存命中率更高,通常更快。

坑点 2:队列/栈场景别用 LinkedList

```
// 不推荐:LinkedList 当队列/栈
LinkedList<Integer> queue = new LinkedList<>();
queue.offer(1);
queue.poll();

// 推荐:用 ArrayDeque
ArrayDeque<Integer> deque = new ArrayDeque<>();
deque.offer(1);
deque.poll();
```

原因:LinkedList 每次操作都要 new Node,对象开销大;ArrayDeque 用循环数组,缓存友好,性能通常好 2-3 倍。

何时用 LinkedList

- 需要频繁头尾操作 + 不在意随机访问 → 可以用(但其实 ArrayDeque 也行)
- 实际刷题场景:优先 ArrayList,其次 ArrayDeque,LinkedList 用得少
- 真要链表结构,常自己写 ListNode 节点类(见 02)

十、pair 展开(Java 无内建 pair)

C++ 的 `pair<T1, T2>` 是个超常用的轻量结构,但 Java 没有内建的 pair 类型。3 种替代写法,按场景选:

方案 1:int[] (2 元素时性能最好)

```
// 2 元素 pair:用 int[2] 数组
int[] p = new int[]{1, 2};
int a = p[0], b = p[1];

// vector<pair<int, int>> → int[][]
int[][] pairs = new int[][]{{1, 2}, {3, 4}, {5, 6}};
for (int[] p : pairs) {
    System.out.println(p[0] + " " + p[1]);
}
```



优点:性能最好,无对象开销。
缺点:语义不清晰,只能按 `p[0]` / `p[1]` 访问,3+ 元素难扩展。

方案 2: `Map.Entry<K, V>` (有 K-V 含义时)

```
import java.util.*;

// 用 Map.Entry 表达 pair
Map.Entry<String, Integer> entry = new AbstractMap.SimpleEntry<>("age", 25);
String key = entry.getKey(); // "age"
Integer val = entry.getValue(); // 25

// 简化:Collections 里也有,但不如 AbstractMap.SimpleEntry 直观
```

优点:有 key-value 语义,可读性好。
缺点:要 import,有对象开销。

方案 3:自定义类 (3+ 字段或需要清晰语义)

```
// 3 字段示例:边(u, v, w)
class Edge {
    int u, v, w;
    Edge(int u, int v, int w) { this.u = u; this.v = v; this.w = w; }
}

List<Edge> edges = new ArrayList<>();
edges.add(new Edge(1, 2, 10));

// 2 字段也可用(追求可读性)
class Pair {
    int first, second;
    Pair(int f, int s) { first = f; second = s; }
}
```

优点:可读性最好,字段名清晰。
缺点:每个 pair 都要 new,有对象开销。

最小代码:邻接表 + 边列表



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 场景 1:边列表 vector<pair<int,int>> → List<int[]>
        int[][] edges = {{0, 1}, {0, 2}, {1, 2}, {2, 3}};
        for (int[] e : edges) {
            int u = e[0], v = e[1];
            System.out.println(u + " -> " + v);
        }

        // 场景 2:邻接表 vector<vector<pair<int,int>>> → List<int[]>[]
        // adj[u] 存 {v, w} 这样的 pair
        int n = 4;
        List<int[]>[] adj = new ArrayList[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        adj[0].add(new int[]{1, 5}); // 0 -> 1, weight 5
        adj[0].add(new int[]{2, 3}); // 0 -> 2, weight 3
        for (int[] e : adj[0]) {
            System.out.println("to " + e[0] + ", w=" + e[1]);
        }

        // 场景 3:有意义的 K-V pair → Map.Entry
        Map.Entry<String, Integer>[] pairs = new Map.Entry[]{
            new AbstractMap.SimpleEntry<>("a", 1),
            new AbstractMap.SimpleEntry<>("b", 2)
        };
        for (Map.Entry<String, Integer> p : pairs) {
            System.out.println(p.getKey() + " = " + p.getValue());
        }
    }
}

```

```

// C++ 对照
#include <vector>
#include <utility>
using namespace std;

int main() {
    // pair<int, int>
    vector<pair<int, int>> edges = {{0, 1}, {0, 2}, {1, 2}};

    // 邻接表
    vector<vector<pair<int, int>>> adj(4);
    adj[0].push_back({1, 5});
    adj[0].push_back({2, 3});

    for (auto& [u, v] : edges) cout << u << " " << v; // C++17
}

```

选型指南



场景	推荐
2 元素 + 性能敏感	int[]
有 K-V 含义	Map.Entry
3+ 字段	自定义类
临时变量 / 不想写类	int[]

实战经验:LeetCode 90% 场景 int[] 就够,写自定义类反而啰嗦。

十一、简讲容器(6 个,一行对照 + 一行代码)

下面 6 个容器刷题时遇到再查,不要死记。

1. Stack:别用,改 ArrayDeque

```
// 错:Stack 继承 Vector,同步开销大
Stack<Integer> s = new Stack<>();

// 正确:用 ArrayDeque
ArrayDeque<Integer> s = new ArrayDeque<>();
s.push(1); s.pop(); // 当栈用
```

2. TreeMap:map<K,V>(红黑树,有序)

```
// 有序 map:O(log n),支持 lowerKey/higherKey/ceilingKey/floorKey
TreeMap<Integer, String> tm = new TreeMap<>();
tm.put(1, "a"); tm.put(3, "c"); tm.put(5, "e");
tm.lowerKey(3); // 1(< 3 的最大 key)
tm.higherKey(3); // 5(> 3 的最小 key)
tm.ceilingKey(3); // 3(>= 3 的最小 key)
tm.floorKey(3); // 3(<= 3 的最大 key)
```

3. TreeSet:set<T>(红黑树,有序)

```
// 有序 set:O(log n),支持 lower/higher/ceiling/floor
TreeSet<Integer> ts = new TreeSet<>();
ts.add(1); ts.add(3); ts.add(5);
ts.lower(3); // 1
ts.higher(3); // 5
ts.ceiling(3); // 3
ts.floor(3); // 3
```

4. Queue(接口):queue<T> → 用 ArrayDeque / LinkedList 实现

```
// Queue 是接口,不能直接 new
Queue<Integer> q = new ArrayDeque<>(); // 推荐
Queue<Integer> q2 = new LinkedList<>(); // 也行,但慢
q.offer(1); q.poll(); q.peek();
```



5. Deque(接口):deque<T> → 用 ArrayDeque 实现

```
// Deque 是接口
Deque<Integer> d = new ArrayDeque<>();
d.addFirst(1); d.addLast(2);
d.peekFirst(); d.peekLast();
d.pollFirst(); d.pollLast();
```

6. Vector(遗留):线程安全版,别用

```
// 错:Vector 同步开销大,过时
Vector<Integer> v = new Vector<>();

// 正确:用 ArrayList
ArrayList<Integer> list = new ArrayList<>();

// 真要线程安全:用 Collections.synchronizedList
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
```

十二、选型决策树(忘记选哪个?翻这里)

```
要栈 / 队列 / 双端队列?
|—— 是 → ArrayDeque(默认,99% 场景)
|
要堆(优先级队列)?
|—— 是 → PriorityQueue(注意:默认小顶堆,需要大顶堆传 Comparator.reverseOrder())
|
要动态数组 / 列表?
|—— 是 → ArrayList(默认,90% 场景)
|
要哈希表(K-V)?
|—— 是 → HashMap
|
要哈希集合(去重 / 存在性)?
|—— 是 → HashSet
|
要有序表(K-V,按 K 排序)?
|—— 是 → TreeMap
|
要有序集合(去重 + 排序)?
|—— 是 → TreeSet
|
要频繁头尾操作 + 不在意随机访问?
|—— 是 → ArrayDeque / LinkedList(优先 ArrayDeque)
|
其他 → 99% 场景上面 6 个已经覆盖,再考虑别的
```

口诀:"栈队双端 ArrayDeque,堆用 PriorityQueue,动态数组 ArrayList,哈希 HashMap/HashSet,有序 TreeMap/TreeSet"。



十三、API 速查总表(本篇所有 API 一行对照)

容器	常用 API
ArrayList<E>	add / get / set / remove(idx) / remove(obj) / size / isEmpty / contains / clear / toArray
ArrayDeque<E>	push / pop / peek / offer / poll / addFirst / addLast / peekFirst / peekLast / pollFirst / pollLast
PriorityQueue<E>	offer / peek / poll / size / new PriorityQueue<>(Comparator.reverseOrder())
HashMap<K,V>	put / get / getOrDefault / containsKey / remove / keySet / values / entrySet / size / isEmpty / clear / computeIfAbsent / merge
HashSet<E>	add / contains / remove / size / isEmpty / clear / addAll / retainAll / removeAll
LinkedList<E>	add / addFirst / addLast / getFirst / getLast / removeFirst / removeLast / offer / poll / peek
TreeMap<K,V>	put / get / lowerKey / higherKey / ceilingKey / floorKey / firstKey / lastKey
TreeSet<E>	add / contains / lower / higher / ceiling / floor / first / last
Queue<T>(接口)	offer / poll / peek / isEmpty
Deque<T>(接口)	addFirst / addLast / pollFirst / pollLast / peekFirst / peekLast
Stack<T>(别用)	用 ArrayDeque 替代
Vector<T>(别用)	用 ArrayList 替代

详细 API 列表(Java 8 新增 getOrDefault / computeIfAbsent / merge 等)见 04 常用工具类。

十四、本篇坑点总结(回到 06 再深讲)

#	坑点	章节	一句话
1	Stack 改用 ArrayDeque	3.2 / 5.1	同步开销大,过时
2	PriorityQueue 默认小顶堆	3.3 / 6.2	反 C++,记得反转
3	Integer 用 == 比较	3.1 / 6.3	缓存池陷阱,永远用 equals
4	遍历时 put / remove	3.4 / 6.4	抛 ConcurrentModificationException
5	Arrays.asList 不能 add/remove	3.1 / 6.5	返回定长 list
6	自定义类不重写 hashCode/equals	3.4 / 3.5	contains/get 永远 false

完整 15 大坑见 06 常见坑点速查,这里只是容器相关的 6 个。



练习建议

- 先把对照表抄一遍(第三节),记忆 12 个映射,做 5 道题后就能默写
 - ArrayList 必做:LeetCode 1 两数之和(用 HashMap 配合)、LeetCode 347 前 K 个高频元素(用 PriorityQueue 配合)
 - ArrayDeque 必做:LeetCode 239 滑动窗口最大值(单调队列)、LeetCode 20 有效括号(当栈用)
 - HashMap 必做:LeetCode 49 字母异位词分组(用 computeIfAbsent 建邻接表)
 - HashSet 必做:LeetCode 349 两个数组的交集(用 retainAll 找交集)
 - LinkedHashMap 必做:LeetCode 146 LRU 缓存(用 new LinkedHashMap<>(capacity, 0.75f, true) 三行实现, O(1) get/put)
 - 简讲容器(Stack / TreeMap / TreeSet 等)遇到再查,不要死记
 - 做完每题都用本篇对照表过一遍,看自己用了哪些容器 API,加深印象
-

下一步:打开 04_常用工具类.md,学 Arrays / Collections / Math / String / StringBuilder,以及 Java 8 必备 API (getOrDefault / computeIfAbsent 等)。

